

Team 2 — ML Engine (Python) Production Roadmap

Model Persistence · Parallel Extraction · Feedback Loop · Benchmarks

ML Team

YourCrawl Dark Pattern Auditor

Generated: 16 August 2026

ML Microservice (Python/FastAPI) Production Roadmap

Assigned To: ML Team Lead

Priority: HIGH — The core intelligence engine of the product

What Exists Today (Current State Assessment)

Component	File	Status
-----------	------	--------

FastAPI Server	api.py	Functional on port 8000
CV Agent	extractors/cv_agent.py	Runs YOLO/CLIP pipeline
NLP Agent	extractors/nlp_agent.py	Transformer-based text analysis
Spatial Agent	extractors/spatial_agent.py	Bounding box / z-index analysis
Screenshot Agent	extractors/screenshot_agent.py	Image-based analysis
Ensemble Classifier	detection/ensemble.py	XGBoost + LightGBM
Heuristic Classifier	detection/heuristic_rules.py	Rule-based fallback
Gemini Gatekeeper	detection/gemini_gatekeeper.py	LLM verification layer
Compliance Mapper	compliance/mapper.py	Maps patterns to regulations
Roadmap Generator	output/roadmap_generator.py	Produces structured tickets
Training Script	train.py	Exists
Async Scan Store	api.py line 125	In-memory dict — CRITICAL RISK
Model Persistence	No model files found	UNKNOWN — models may not be saved
Input Size Validation	None	MISSING
Model Versioning	None	MISSING
Feedback Loop	None	MISSING
Performance Benchmarks	None	MISSING

Critical Flaws Found

1. Ensemble Models May Not Be Trained / Loaded from Disk

Files: `detection/ensemble.py`, `train.py` The `EnsembleClassifier` class uses XGBoost and LightGBM. However, there are no `.pkl` or `.model` files visible in the repo structure. This raises a critical question: **are the models being trained fresh on every startup, or loaded from saved weights?** If trained fresh on startup using the `mathur_dataset.tsv` file, this is extremely slow (30-60 seconds per startup) and produces non-deterministic results across instances. **Fix:** Train models once offline, save them using `joblib.dump()`, and load saved weights at startup.

2. In-Memory Scan Store (State Loss on Restart)

File: `api.py` line 125

```
scan_store: dict[str, ScanStatusResponse] = {}
```

Any in-flight async scan is lost permanently if the process restarts. Users polling for results will get 404 with no explanation. **Fix:** Replace with Redis (coordinated with Backend Team — Feature 6 in their document).

3. No Input Size Validation (Denial of Service Risk)

The `POST /api/v1/scan` endpoint accepts `CrawlData` which can theoretically contain thousands of elements per page and multiple pages. A malicious or buggy crawl could send megabytes of data and cause:

- Memory exhaustion
- Extremely long processing times
- Out-of-memory crashes (especially with CLIP/transformer models loaded)

Fix: Add Pydantic validators for maximum elements per page, maximum pages, and maximum text lengths.

4. Feature Extraction Is Sequential (Slow)

File: `main.py` lines 84-99

```
cv_features = cv_agent.analyze(page)    # ~2-5s
nlp_features = nlp_agent.analyze(page)  # ~1-3s
spatial_features = spatial_agent.analyze(page) # ~0.5s
```

CV, NLP, and Spatial agents run one after another even though they have no interdependency. Total extraction time is the sum of all three instead of the maximum. **Fix:** Run all three agents in parallel using

`asyncio.gather()` or `ThreadPoolExecutor`.

5. No Model Versioning or Registry

When the ensemble model is retrained on new data, there is no way to:

- Track which version of the model produced which audit results
 - Roll back to a previous model if a new one performs worse
 - A/B test two model versions
-

6. No Feedback Loop for Model Improvement

The Backend saves training records to `training_data/` JSON files. However, there is no pipeline to:

- Ingest these new training records into the model
- Retrain the ensemble automatically
- Evaluate performance before deploying the new model
- Label false positives reported by users

Without a feedback loop, the model accuracy will stagnate.

7. CORS Wildcard in Production

File: `api.py` line 64

```
allow_origins=["*"],
```

The ML service allows CORS from any origin. In production, this service should only be accessible by the Backend service, not from browsers directly.

System Architecture: Target State

```
Backend Node.js Service
|
v
[ML FastAPI - port 8000]
|
+--[CV Agent]-----+
|                   |-- (parallel) --> [Feature Bundle]
+--[NLP Agent]-----+                   |
|                   |                   v
+--[Spatial Agent]---+                   [Ensemble Classifier]
|                   |                   |
|                   |                   [Heuristic Rules Fallback]
|                   |                   |
|                   |                   [Gemini Gatekeeper (optional)]
|                   |                   |
|                   |                   [Compliance Mapper]
|                   |                   |
|                   |                   [Roadmap Generator]
|                   |                   |
|                   |                   Roadmap JSON Response

[Redis]      <->  Async Scan Store (persistent)

[Model Registry] <- saved .pkl files, versioned

[Feedback DB]  <- user-reported false positives
```

Feature Tasks to Build

Feature 1 — Train and Persist Ensemble Models (HIGHEST PRIORITY)

Why: Without trained, persisted model weights, every restart retriggers training which is slow and non-deterministic. **Implementation Steps:**

11. Verify whether `train.py` currently saves models. If not, add:

```
import joblib

joblib.dump(xgb_model, 'models/xgb_v1.pkl')

joblib.dump(lgb_model, 'models/lgb_v1.pkl')
```

12. In `ensemble.py`, load from disk at startup:

```
def load_model(self):

    model_path = os.getenv('MODEL_PATH', 'models/xgb_v1.pkl')

    if os.path.exists(model_path):

        self.model = joblib.load(model_path)

        logger.info(f"Loaded model from {model_path}")

    else:

        logger.warning("No saved model found - training from scratch")

        self.train()
```

13. Create a `models/` directory and commit the trained model files.

14. Add `MODEL_PATH` to `.env`.

Acceptance Criteria: The ML service starts in under 5 seconds by loading pre-trained weights. The same model version produces identical results for the same input.

Feature 2 — Parallel Feature Extraction

Why: Running CV + NLP + Spatial sequentially wastes 3-8 seconds per page that could be saved through parallelism. **Implementation Steps:**

```
import asyncio

from concurrent.futures import ThreadPoolExecutor

executor = ThreadPoolExecutor(max_workers=3)

async def extract_features_parallel(page, use_clip, use_transformer):

    loop = asyncio.get_event_loop()

    cv_task = loop.run_in_executor(executor, cv_agent.analyze, page)
    nlp_task = loop.run_in_executor(executor, nlp_agent.analyze, page)
    spatial_task = loop.run_in_executor(executor, spatial_agent.analyze, page)

    cv_features, nlp_features, spatial_features = await asyncio.gather(

        cv_task, nlp_task, spatial_task

    )

    return FeatureBundle(cv=cv_features, nlp=nlp_features, spatial=spatial_features)
```

Update `run_pipeline()` in `api.py` to call `extract_features_parallel()`. **Acceptance Criteria:** Feature extraction for a page with 200 elements completes in under 3 seconds (down from 8-10 seconds).

Feature 3 — Input Size Validation

Implementation Steps: Add Pydantic validators to `schemas/input_schema.py`:

```
from pydantic import validator

class PageData(BaseModel):
    elements: list[ElementData]

    @validator('elements')
    def check_element_count(cls, v):
        if len(v) > 2000:
            raise ValueError('Too many elements: maximum 2000 per page')
        return v

class CrawlData(BaseModel):
    pages: list[PageData]

    @validator('pages')
    def check_page_count(cls, v):
        if len(v) > 10:
            raise ValueError('Too many pages: maximum 10 per request')
        return v
```

Acceptance Criteria: A request with more than 2000 elements per page is rejected with HTTP 422 and a clear error message.

Feature 4 — Persistent Async Scan Store (Redis)

Why: In-memory `scan_store` dict loses all state on restart. **Implementation Steps:**

15. Install: `pip install redis`

16. Create `storage/scan_store.py`:

```

import redis, json

from datetime import datetime

r = redis.Redis(host=os.getenv('REDIS_HOST', 'localhost'), decode_responses=True)

class ScanStore:

    TTL = 3600 # 1 hour

    def save(self, scan_id: str, data: dict):

        r.setex(f"scan:{scan_id}", self.TTL, json.dumps(data, default=str))

    def get(self, scan_id: str) -> dict | None:

        val = r.get(f"scan:{scan_id}")

        return json.loads(val) if val else None

    def update_status(self, scan_id: str, status: str, **kwargs):

        data = self.get(scan_id) or {}

        data.update({'status': status, 'updated_at': datetime.now().isoformat(), **kwargs})

        self.save(scan_id, data)

```

17. Replace all `scan_store[scan_id]` references in `api.py` with `ScanStore()` calls.

Acceptance Criteria: Async scan results survive process restarts. Polling returns status correctly after restart.

Feature 5 — Model Versioning and A/B Testing

Implementation Steps:

18. Add version metadata to saved models:

```

model_metadata = {

    'version': '1.0.0',

    'trained_at': datetime.now().isoformat(),

    'training_samples': len(X_train),

    'accuracy': accuracy_score(y_val, y_pred),

    'features': feature_names

}

joblib.dump({'model': xgb_model, 'metadata': model_metadata}, 'models/xgb_v1.0.0.pkl')

```

19. Add `GET /api/v1/model/info` endpoint that returns current model version, training date, and accuracy metrics.

20. Support loading a specific model version via env var: `MODEL_VERSION=1.2.0`

Acceptance Criteria: `GET /api/v1/model/info` returns version, trained_at, and accuracy metrics.

Feature 6 — Feedback Loop and Continuous Improvement

Why: Without this, the model accuracy is frozen at whatever the initial training achieved. **Implementation**

Steps:

21. Add `POST /api/v1/feedback` endpoint:

```
class FeedbackRequest(BaseModel):  
    scan_id: str  
    ticket_id: str  
    is_false_positive: bool  
    correct_label: Optional[str] = None  
    user_comment: Optional[str] = None
```

22. Save feedback to a `feedback_log.jsonl` file or database table.

23. Create a weekly retraining script `scripts/retrain_with_feedback.py` that:

- Loads all feedback records
- Merges with original training data
- Retrains the ensemble
- Evaluates against a held-out test set
- Saves new model only if performance improves

24. Wire feedback endpoint to Backend's audit report — "Flag as incorrect" button.

Acceptance Criteria: False positives reported by users are collected and available for the next retraining cycle.

Feature 7 — Performance Benchmarking and Profiling

Implementation Steps:

25. Add timing instrumentation to each pipeline stage:

```
import time  
  
stage_times = {}  
  
t0 = time.perf_counter()  
  
cv_features = cv_agent.analyze(page)  
  
stage_times['cv_extraction'] = time.perf_counter() - t0
```

26. Return `processing_time_ms` breakdown in the Roadmap response.

27. Create `stress_test.py` (already exists — extend it) with:

- 10 concurrent requests
- Measure p50, p95, p99 latency
- Track memory usage

28. Set performance targets: p95 latency under 10 seconds for a 200-element single-page scan.

Acceptance Criteria: API response includes `processing_time_ms` per stage. Stress test passes at 10 concurrent users.

Feature 8 — CORS Hardening and API Security

Implementation Steps:

29. Replace wildcard CORS with explicit origin list:


```

app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.getenv('ALLOWED_ORIGIN', 'http://localhost:3000')],
    allow_credentials=True,
    allow_methods=["POST", "GET"],
    allow_headers=["X-API-Key", "Content-Type"],
)

```

30. Rotate the `API_KEY` from the default value `changeme` :

```

API_KEY = os.getenv("API_KEY")
if not API_KEY or API_KEY == "changeme":
    raise RuntimeError("ML_API_KEY not set - refusing to start")

```

31. Add request size limits using `app.add_middleware` with a body size cap.

Acceptance Criteria: ML service rejects all requests from origins other than the Backend. Service fails to start if API key is not set.

Model Accuracy Requirements

Before the product can launch, the ML ensemble must meet minimum accuracy targets on a held-out test set:

Metric	Minimum Target
Overall Accuracy	$\geq 80\%$
Precision (avoid false positives)	$\geq 75\%$
Recall (catch real patterns)	$\geq 70\%$
F1 Score	$\geq 72\%$

These baselines should be computed using the `mathur_dataset.tsv` file and documented before launch.

Delivery Timeline

Week	Milestone
Week 1	Feature 1 (Train + Persist Models) + Feature 3 (Input Validation)
Week 2	Feature 2 (Parallel Extraction) + Feature 4 (Redis Scan Store)
Week 3	Feature 5 (Model Versioning) + Feature 8 (Security Hardening)
Week 4	Feature 6 (Feedback Loop) + Feature 7 (Performance Benchmarks)

New Dependencies to Add

```
redis>=5.0  
joblib>=1.3  
pytest>=8.0  
pytest-asyncio>=0.23  
locust>=2.x (load testing)
```

Definition of "Production Ready" for ML Service

- ☐ Ensemble models trained, saved to disk, and loaded at startup in under 5 seconds
- ☐ Parallel feature extraction working
- ☐ Input size validation active
- ☐ Redis-backed scan store (no in-memory dict)
- ☐ Model version info available via API
- ☐ API key hardcoded default `changeme` removed
- ☐ CORS restricted to Backend origin only
- ☐ Performance target: p95 latency under 10 seconds per page
- ☐ Feedback collection endpoint working
- ☐ Accuracy metrics documented and meeting targets